

SESSION-8
MODULE-5

```
-- =====  
  
-- Session 8 — MySQL equivalent of the PL/SQL tasks  
-- (Use this if you're staying in phpMyAdmin/MySQL rather than  
-- switching to Oracle Live SQL.)  
--  
-- MySQL doesn't have free-standing DECLARE/BEGIN/END blocks like  
-- Oracle — procedural logic only exists INSIDE a stored procedure.  
-- So each task below is wrapped in CREATE PROCEDURE ... CALL ... DROP.  
--  
-- The DELIMITER trick: MySQL normally ends a statement at ';'.  
-- But our procedure body is FULL of semicolons internally, so we  
-- temporarily change the statement terminator to // while defining  
-- it, then switch back to ; afterwards.  
-- =====  
  
USE instaclone;  
  
-- ----- Task 1: two variables, printed -----  
  
DELIMITER //  
DROP PROCEDURE IF EXISTS task1_vars //  
CREATE PROCEDURE task1_vars()  
BEGIN  
    DECLARE v_user_name VARCHAR(50);  
    DECLARE v_age INT;  
  
    SET v_user_name = 'Priya Mehta';  
    SET v_age = 27;
```

SESSION-8
MODULE-5

```
SELECT v_user_name AS name, v_age AS age;

END //

DELIMITER ;

CALL task1_vars();

-- ----- Task 2: IF on order amount -----

DELIMITER //

DROP PROCEDURE IF EXISTS task2_delivery_check //

CREATE PROCEDURE task2_delivery_check(IN p_order_amount DECIMAL(10,2))

BEGIN

    IF p_order_amount > 500 THEN

        SELECT 'Eligible for free delivery' AS result;

    ELSE

        SELECT 'Delivery charges apply' AS result;

    END IF;

END //

DELIMITER ;

CALL task2_delivery_check(650); -- try CALL task2_delivery_check(300); too

-- ----- Task 3: loop printing 1 to 5 -----

-- MySQL has WHILE/REPEAT/LOOP inside procedures. Each printed number
-- is collected into a temp table so all 5 show up as one result set
-- (a procedure can't just PUT_LINE five separate times the way
-- PL/SQL does — this is the practical MySQL workaround).
```

SESSION-8
MODULE-5

```
DELIMITER //

DROP PROCEDURE IF EXISTS task3_count_to_5 //

CREATE PROCEDURE task3_count_to_5()

BEGIN

    DECLARE v_counter INT DEFAULT 1;


    DROP TEMPORARY TABLE IF EXISTS tmp_counter_output;
    CREATE TEMPORARY TABLE tmp_counter_output (n INT);


    WHILE v_counter <= 5 DO

        INSERT INTO tmp_counter_output VALUES (v_counter);

        SET v_counter = v_counter + 1;

    END WHILE;


    SELECT * FROM tmp_counter_output;

END //

DELIMITER ;


CALL task3_count_to_5();
```

-- ----- Task 4: cursor over PRODUCTS.PRODUCT_NAME -----

```
DROP TABLE IF EXISTS PRODUCTS;

CREATE TABLE PRODUCTS (

    PRODUCT_ID INT AUTO_INCREMENT PRIMARY KEY,

    PRODUCT_NAME VARCHAR(100)

);
```

SESSION-8
MODULE-5

INSERT INTO PRODUCTS (PRODUCT_NAME) VALUES

('Wireless Earbuds'), ('Smart Watch'), ('Bluetooth Speaker');

DELIMITER //

DROP PROCEDURE IF EXISTS task4_list_products //

CREATE PROCEDURE task4_list_products()

BEGIN

DECLARE v_product_name VARCHAR(100);

DECLARE v_done INT DEFAULT 0;

-- The cursor itself

DECLARE cur_products CURSOR FOR

SELECT PRODUCT_NAME FROM PRODUCTS;

-- Tells MySQL what to do when the cursor runs out of rows

DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;

DROP TEMPORARY TABLE IF EXISTS tmp_product_output;

CREATE TEMPORARY TABLE tmp_product_output (name VARCHAR(100));

OPEN cur_products;

read_loop: LOOP

FETCH cur_products INTO v_product_name;

IF v_done = 1 THEN

LEAVE read_loop;

END IF;

INSERT INTO tmp_product_output VALUES (v_product_name);

SESSION-8
MODULE-5

END LOOP;

CLOSE cur_products;

SELECT * FROM tmp_product_output;

END //

DELIMITER ;

CALL task4_list_products();

-- ----- Task 5: division with exception handling -----

DELIMITER //

DROP PROCEDURE IF EXISTS task5_safe_divide //

CREATE PROCEDURE task5_safe_divide(IN p_num1 DECIMAL(10,2), IN p_num2
DECIMAL(10,2))

BEGIN

DECLARE v_result DECIMAL(10,4);

-- MySQL doesn't throw a catchable error on x/0 by default — it

-- just returns NULL with a warning. So we check explicitly instead

-- of relying on an EXCEPTION block the way Oracle does.

IF p_num2 = 0 THEN

SELECT 'Cannot divide by zero' AS result;

ELSE

SET v_result = p_num1 / p_num2;

SELECT v_result AS result;

END IF;

SESSION-8 MODULE-5

END //

DELIMITER ;

CALL task5_safe_divide(100, 0); -- prints the error message

CALL task5_safe_divide(100, 4); -- prints 25.0000

The image consists of two screenshots of the phpMyAdmin web interface.

The top screenshot shows the 'Structure' tab for a table named 'products'. The 'Parameters' section shows a parameter 'p_order_amount' of type 'DECIMAL' with a length of '10,2'. The 'Definition' section shows the following SQL code:

```
BEGIN
  IF p_order_amount > 500 THEN
    SELECT 'Eligible for free
delivery' AS result;
  ELSE
    SELECT 'Delivery charges apply' AS
result;
  END IF;
END
```

The bottom screenshot shows the 'Browse' tab for the 'products' table. It displays the following data:

PRODUCT_ID	PRODUCT_NAME
1	Wireless Earbuds
2	Smart Watch
3	Bluetooth Speaker